

Data handling using Python

12

12.1 Introduction

Data handling is the process of collecting, storing and analyzing data. Data handling is a key skill for biologists, as understanding and processing data is integral to their work. Data handling is often associated with statistical analysis, which can be subjective or quantitative (Marx, 2013). Statistical analysis can often be used to answer questions about what qualifies as statistically significant and how large samples are necessary for accurate conclusions. Data handling begins with the data producer defining (1) what the type of data is (e.g., genetic, behavioral) and (2) the question to answer based on that type of data: for example, how many animals there are in a population on an island? The next step would be to choose an appropriate sampling method: random sampling, systematic sampling, or opportunistic sampling. The data producer then (3) gathers the data, records it in a spreadsheet or database, and then (4) analyzes and publishes their analysis. Data management is the process that ensures that raw and processed data can be organized efficiently in order to investigate specific questions related to the experimental task at hand (Hasija and Chakraborty, 2021).

In bioinformatics research, a variety of programming languages are often utilized, including Python, R, C, and Java. Python-based bioinformatics research is being introduced in this project. Guido van Rossum developed this all-purpose programming language, which was first made available in 1991. Python is a great language to learn for beginners into coding. Few basic characteristics of Python are:

Interpreted language: So, Python is a Interpreted Language as it itself is a software running on the computer. A Python script is just a text file when it is written. Python program may be instructed to do a variety of tasks on a text file, such as operations on genome's sequence or creating graphs etc. As opposed to compiled languages like C, which transforms the code into an executable. Generally speaking, compiled languages may operate more quickly than interpreted ones. The ability to quickly test every iteration of a script without having to do an additional compilation step, however, helps speed up the creation of new applications. A tradeoff between development time spent developing a code time the computer spends actually running your code. Because the development time is quite lengthy relative to the run time in many scientific applications where the user base is tiny and new code must be regularly produced, interpreted languages are often employed.

High-level language: High level languages include a lot of built-in automated procedures that take care of various elements of how the code is written is used by a computer. For instance, Python takes care of allocating memory to keep the contents of the variable when it is defined, so user don't have to. In contrast, manual memory allocation is required for other languages like C. In general, writing a functioning program rapidly in a high-level language is substantially simpler. Low-level languages, on the other hand, may provide fine-grained control that can aid in the optimization of extremely computationally demanding applications.

Python has become one of the most popular programming languages in recent years. Its versatility and ease of use make it a great choice for a wide range of tasks, including data handling. Python has become the language of choice for data science and machine learning due to its simple syntax and powerful libraries. These libraries allow Python to be used for anything from web development to scientific computing. Python is especially popular in the field of data science. Some of the most popular Python libraries for data science are Pandas and NumPy. These libraries allow you to easily manipulate data and perform statistical operations ([Ekmekci et al., 2016](#)).

BeOpen Python Labs created the first version of Python 2.0, which was released in the year 2000. Prior to the formation of the team, Rossum (conceiver of Python) was in charge of the majority of Python's new features and bug fixes ([van Rossum and Drake, 2009](#)). He remained hopeful that Python would play a larger role in increasing computer science "literacy." Python was designed so that even someone with no prior knowledge of computers can use it. As a result, the Python Labs team created Python 2.X to make it less reliant on Rossum's management and more accessible to community contributions. Python 2.7 was the final and most recent version of Python 2. The Python 2 was discontinued in 2020. Even though Python 3.0 was released in 2008, it was more than just a bug-fixing version of Python 2. Instead, it introduced a significant change that rendered the language incompatible with subsequent editions. One purpose of the Python 3 syntax reform was to allow code to achieve the same thing in multiple ways ("[Python 3.0 Release | Python.org](#)," n.d.). The purpose of Python 3.X is to ensure that everyone utilizes the same, simple way. This eliminates the most common issues that novice programmers have when learning a new language. Python is a community-driven programming language that is widely used in the field of data science and machine learning. Because of its relatively flat learning curve, many experts recommend it as the first programming language to learn for beginners. Primarily, Python has an easy-to-understand English-like readable syntax.

12.2 Datatypes and operators

12.2.1 Datatypes

Data structures are specialized methods for storing and arranging information while programming. Each offers various methods of dealing with the data and is best

suited for certain sorts of data. Gaining proficiency with Python's data types and understanding when to utilize each will help you develop into a professional Python programmer. Among the several datatypes, the four most prevalent are as follows:

- int (integers or whole numbers)
- float (decimal numbers or floating-point numbers)
- bool (Boolean or True/False)
- str (string or a collection of characters like a text)
- list, Tuple, set (Collection of Items)

Python has two significant ways of representing numbers: int and float. Decimal values (floats) such as 1.0, 3.14, and -2.33 may take up more space than integers or whole numbers such as 1, 3, -4 , and 0. Following that, there are Boolean datatype that is either "True" or "False"; these are utilized to create conditions, discussed in conditional statements. Finally, the "str" or string datatype is the most frequently used and encountered by biologists, as the majority of DNA, RNA, protein sequences, and names are text or strings. As a result, this chapter contains a distinct section on strings. It is critical to emphasize here that string data is always enclosed in quotes, i.e., ("`<string data>`"). For instance the peptide "MKSGSGGGSP" will be a Python string.

12.2.2 Operators

There are common operators as used like "+", "-", "*", "/", "=", and "**" for addition, subtraction, multiplication, division, assignment and exponent respectively.

Operation on an integer and a float will always produce a float type result, while operation on two integers will always return an integer type result, with the exception of division, which will always return a float type result. By employing the integral division operator, one can obtain an integer type in exchange for division. i.e., "//".

12.3 Variables

Variables in Python are analogous to algebraic variables in mathematics. Variables are composed of two components: a name and a value. Variables names declared and assigned a name by using the assignment "=" operator. On the left is the name, and on the right is the value (Table 12.1).

Variables can be recalled after they have been assigned. As seen in Table 12.2, variables "weight" in kilogrammes and "height" in meters are assigned and then utilized to calculate BMI, which is subsequently stored in another variable named "bmi."

Variables make our programmes readable and reusable. For example, if one is working with a large protein or nucleotide sequence, it would be imprudent to write it every time. As a result, it can be saved in a variable and reuse it whenever it is

Table 12.1 Assigning variables in Python.

Code	Output
<pre>weight = 75 print(weight)</pre>	75

Table 12.2 Operations with variables.

Code	Output
<pre>weight = 75 height = 1.5 bmi = weight/height print(bmi)</pre>	50.0

required. Variables can be assigned to other variables, reassigned to new values at any moment, and even assigned to another variable. When a new value is assigned to a variable, the previous value is lost and cannot be recovered. This reassignment of a variable is also possible when the data type is not the same. For instance, an integer variable can be reassigned to a text variable and vice versa. This is not true for most of the other programming languages. The last sentence assigns values to two variables in the same statement, which is a feature that is rarely found in any other programming language. Finally, variable names are case sensitive; for example, the variable name “gene_symbol” cannot be recall as “Gene Symbol” or “GENE SYMBOL.”

12.4 Strings

For computer programmers, strings are the collection of characters or, more commonly, any texts. String manipulation is quite prevalent in bioinformatics investigations, such as sequence files, pattern discovery in sequences, data mining from texts, and data processing from a variety of file types. In Python, a string object can be formed by containing a sequence of characters within a pair of single quotes/double quotes/triple single quotes/triple-double quotes. While characters

included in single or double quotation marks can only contain a single line, characters wrapped in triple single or triple-double quotation marks can contain many lines. The characters should be enclosed within the same type of quote—usually single or double quotes for defining a string datatype. Consider the following example (Table 12.3).

12.4.1 String indexing

As a string is a collection or sequence of characters, Python allows for the extraction of individual characters as well as portions of the text via their indexes. To retrieve the character, the index number must be placed following the string variable inside the square bracket pair.

The following is an example of using String indexing to print the second character in the word “PLANT.”

It’s worth noting that the index of any string begins with 0, beginning with the leftmost character; so, the index of the first character “P” is 0; the index of the second character “L” is 1, and so on. Backward indexing begins with -1 from the rightmost character, indicating that the last character “T” has a backward index of -1 , the second last character “N” has a backward index of -2 , and so on.

Another example of character index for Python is shown in Table 12.4, where first row is the sequence, the second row is the forward index of nucleotides, and the third row shows the backward index:

Table 12.3 Defining strings in Python.

Code	Comments	Output
<code>seq_1 = 'MALNSGSPPA'</code> <code>print(seq_1)</code>	A string within a pair of single quotes	MALNSGSPPA
<code>seq_2 = "MALNSGSPPA"</code> <code>print(seq_2)</code>	A string within a pair of double quotes	MALNSGSPPA
<code>seq_3 = '''MALNSGSPPA'''</code> <code>print(seq_3)</code>	A string within a pair of triple single quotes	MALNSGSPPA
<code>seq_4 = """MALNSGSPPA"""</code> <code>print(seq_4)</code>	A string within a pair of triple double quotes	MALNSGSPPA
<code>seq_5 = '''MALNSGSPPA IGPYENHGY'''</code> <code>print(seq_5)</code>	A string within a pair of triple single quotes, can have multiple lines	MALNSGSPPA IGPYENHGY
<code>seq_6 = """IGPYENHGY IGPYENHGY"""</code> <code>print(seq_6)</code>	A string within a pair of triple double quotes, can have multiple lines	IGPYENHGY IGPYENHGY

To take a portion of a text or string, use the annotation “string_name [start: end]”, where the start is the starting index and the end is the index extending up to but not including it.

- `word[1:3]` is “LA”—characters starting at index 1 and extending up to but not including index 3
- `word[3:]` is “NT”—leaving a blank for either index defaults to the start or end index of the string
- `word[:]` is “PLANT”—emptying both always gives us a copy of the whole thing.
- `word[1:10]` is “LANT”—an index that is too big is truncated to string length.
- `word[:-2]` is “PLA”—selecting up to but not including the last 4 chars.
- `word[-2:]` is “NT”—starting with the fourth character from right end to the right end.

12.4.2 Operations on strings

There are a few ways to concatenate or join strings. The easiest and most common way to add join strings is to use the plus symbol (+). i.e., in simplest terms, merely adding them.

The “+” operator can be used to combine any number of strings. A critical point to remember is that when adding strings, all datatypes must be strings; for example, if users add a string with an integer, such as “PLANT”+4, an error message indicates that the “str” type and the “int” type cannot be added. To add a number, it must first convert it to the “str” type using the `str (number)` function. While integers and strings cannot be added, the same string can be printed several times using the “*” operator and a “int” datatype. For instance, “PLANT”*2 returns the string twice, i.e., “PLANTPLANT”.

12.4.3 Methods in strings

Several string handling methods include `count ()`, `find ()`, and `len ()`. Their application is outlined below in [Table 12.5](#).

In the above instances, “`len ()`” is a function that returns the string’s length. There is a key method called `str.split ()` which is regularly been used to extract data from delimited text file formats such as CSV, TSV, and others. CSV stands for comma-separated values, in which each column’s values are separated by a comma, while TSV means for tab-separated values, in which each column’s values are separated by a tab delimiter.

Table 12.4 String indexing in Python.

	P	L	A	N	T
Forward indexing	0	1	2	3	4
Backward indexing	−5	−4	−3	−2	−1

Table 12.5 Few methods in strings.

Code	Output
<pre>peptide = 'TSLWGLLFLSAALSLWPTSG' print(peptide.count('A')) print(peptide.find('LW')) print(len(peptide))</pre>	<pre>2 2 20</pre>

```
sepal_length,sepal_width,petal_length,petal_width,species
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.9,2.4,3.3,1.0,Iris-versicolor
5.9,3.0,5.1,1.8,Iris-virginica
```

FIG. 12.1

Example of a CSV formatted file.

Fig. 12.1 is an example of CSV formatted file where the first row is known as header row, which consists of column names, and the rest of the rows are instances having values separated by a comma for each column. The values can be extracted from each row if each row is considered as a string using `str.split()` method:

In Table 12.6 the first observation, or the second row of the csv file in Fig. 12.3, is assigned to a variable named “first row”. Then Python’s many variables assignment feature was used to establish each column as a variable and the first observations as its value. The `split(“,”)` method returns a list of values separated by commas. The variables named after the CSV file’s columns can be printed. The output also includes a list of split values. List is a Python data type that will be covered in the next section. There are additional intriguing methods available for the string data-type, which can be found in the Python documentation.

12.5 Python lists and tuples

After learning about datatypes like integers, strings, and booleans, this section will cover Lists.

Lists, like containers, store many values of any type. They’re called data structures because they store data in a way that makes retrieval easy.

Lists are similar to arrays in other programming languages, but they are more versatile. List items are called elements. Lists have crucial features like:

- List keeps track of the items inserted and can be accessed later.
- Index access—a list’s objects can be indexed.
- Lists can contain any entity—numbers, strings, and even other lists.
- Lists can be mutated—New items can be added, and old ones can be removed or altered.

Table 12.6 String’s split () method.

Code	Output
<pre>#str.split() first_row = '5.1,3.5,1.4,0.2,Iris-setosa' sepal_length,sepal_width,petal_length,petal_widt h,species = first_row.split(',') print(sepal_length) print(sepal_width) print(petal_length) print('-----') print(first_row.split(','))</pre>	<pre>5.1 3.5 1.4 ----- ['5.1','3.5','1.4', '0.2','Iris-setosa']</pre>

12.5.1 Accessing values in list

Like strings, list items also have indexes starting with “0” for forward indexing and “-1” for backward indexing (Fig. 12.2).

The items inside a list can be accessed using brackets [] and indexes.

Slicing a list allows to access a subset of it. The string slice operator can also slice lists. Similarly, to string, omitting the first index causes the slice to begin at the beginning. If the second is absent, the slice ends. If both of them are removed, the slice is a copy of the List (Table 12.7).

The “+” operator can be used to concatenate two lists and the “*” operator to repeat a list any specified number of times.

	['Moss']	'Embryophyte'	'Thallophyte'	'Conifer']
Forward indexing	0	1	2	3
Backward Indexing	-4	-3	-2	-1

FIG. 12.2

Python list indexes.

Table 12.7 List slicing.

Code	Output
<pre># not including index 2 print(plants[0:2]) # everything up to index 3 print(plants[:3]) # index 1 to end of list print(plants[1:]) # Coping whole list print(plants[:])</pre>	<pre>['Moss', 'Embryophyte'] ['Moss', 'Embryophyte', 'Thallophyte'] ['Embryophyte', 'Thallophyte', 'Conifer'] ['Moss', 'Embryophyte', 'Thallophyte', 'Conifer']</pre>

12.5.2 Methods with lists

Python provides some in-built methods for List such as:

- count () methods will return the total number of occurrences of an item in the List.
- index () will give the index of an item.
- append () adds an item at the end of the List.
- remove () will remove the first occurrence of the item in the List.
- pop () will remove the item at index provided by the user.
- min (), max () and sum () will provide the minimum, maximum and sum of the lists constituting number values.
- len () will provide the total number of items in the List.
- sort () method can be used to sort a list of numerical values in increasing or decreasing order, or a list of string in A-Z or Z-A order.

12.5.3 Tuples

A “tuple” is one of Python’s four built-in data types for storing collections. Tuples’ components, unlike those of other data types, are both sequential and immutable. They can be used to keep track of a variety of different items in a single variable, and they come with a number of useful actions pre-programmed in. There are numerous parallels between these tuples and lists. Developers who are familiar with lists, a commonly used data structure, may mistake lists for tuples. Tuples in Python, like lists, are collections of elements of any data type, but unlike lists, tuples are immutable; that is, once assigned, users cannot change the tuple’s contents or the tuple itself.

12.6 Dictionary in Python

Dictionaries are data structures in Python that are similar to hash tables or hashmaps in other computer languages. Each key corresponds to a single value in a dictionary. The ideal approach to establish a dictionary is to put the key:value pairs inside curly brackets “{}”. Only “{}” can declare an empty dictionary (Fig. 12.3) (Table 12.8).

The Python dictionary has the following properties:

- Dictionaries are unordered—the key-value pairs are not sorted.
- Dictionaries are mutable, which means they can be expanded or trimmed as needed.
- The dictionary values can be access by keys. Indexes can’t be accessed because key-value pairs aren’t kept in order.
- A dictionary can only contain unique keys. The keys in the dictionary are always unique.
- Dictionary keys, such as strings, integers, and tuples, should be immutable data types.

Python dictionary values can be numbers, texts, lists, or even dictionaries. A nested dictionary is a dictionary inside a dictionary. The key in square bracket retrieves the dictionary’s stored values.

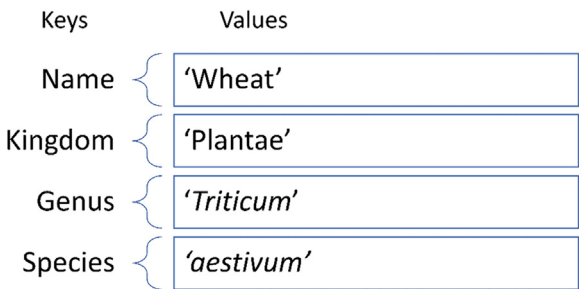


FIG. 12.3

Python dictionary key: Value pairs.

Table 12.8 Creating a Python dictionary.

Code	Output
<pre>crop = {} crop {'Name': 'Wheat', 'Kingdom': 'Plantae', 'Genus': 'Triticum', 'Species': 'aestivum'} print(crop) print(type(crop))</pre>	<pre>{'Name': 'Wheat', 'Kingdom': 'Plantae', 'Genus': 'Triticum', 'Species': 'aestivum'} <class 'dict'></pre>

12.7 Conditional statements

Until now, the programmes are simple, not clever, and not making decisions. Conditional statements are required to make a program make decisions based on conditions. Computers have only two states, True or False, like a light switch has two states, On or Off. In Python, these True/False situations are known as booleans.

A condition is always defined by comparison, such as larger than, less than, or equal. Here are some comparisons with Python operators:

- Equal: `a == b`
- Not Equal: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

All these comparisons result in Boolean values “True” or “False”.

12.7.1 Logical operators

When comparing many conditions, logical operator are used. The logical operators “and”, “or”, and “not” are the same in Python as in English. Logic operators usually work on conditions.

“and” operator will only give true if both the conditions are true, “or” will give true if either of the conditions is true, lastly “not” will give just the opposite condition, i.e., it will give false for true and true for false.

12.7.2 If and else statements

Often, it is required to execute some statement only if some conditions are true. For this, “if” and “else” statements are used.

1. “If” Statement: use it to execute a block of code if the specified condition is true.
2. “Else” statement: use with an “if” statement to execute a block of code if the specified condition is false.

In Python, blocks are defined by indentation after a colon. Indentation is a strict Python syntax for forming a statement block, which can be commonly encounter. If the condition is true, an indented block of statements will be executed (see [Fig. 12.4](#)). Let’s see an example of comparing the expression of a gene in a controlled and treated environment in [Table 12.9](#).

Next example is for comparing the expression of a gene in a controlled and treated environment, with “If” and “else” statements ([Table 12.10](#)).

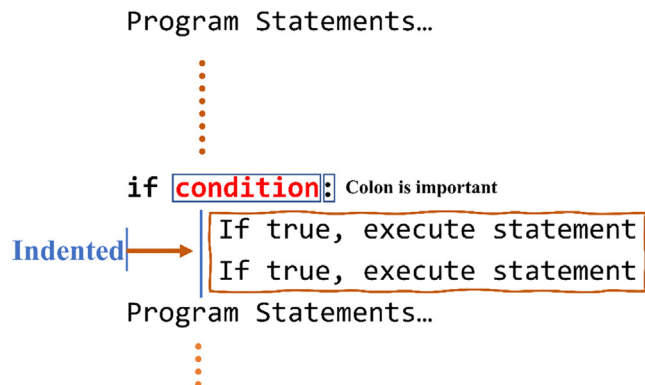


FIG. 12.4

Syntax of “if” statement.

Table 12.9 If statements.

Code	Output
<pre>control_expression = 14 treated_expression = 3.5 if control_expression treated_expression: print('downregulated')</pre>	<pre>> downregulated</pre>

Table 12.10 If, else statement.

Code	Output
<pre>control_expression = 14 treated_expression = 3.5 if control_expression > treated_expression: print('Gene is downregulated') else: print('Gene is upregulated')</pre>	<pre>Gene is downregulated</pre>

12.8 Loops in Python

When developing code, it is occasionally necessary to repeat a block of code or a statement until the condition is met or the statement is false (depend on requirement). This type of recurring or iterative behavior can be implemented using loops. These are known as control structures, and programming languages offer a variety of options to facilitate their implementation.

Generally, there are two type of loop structure:

1. While Loop
2. For Loop

12.8.1 While loop

In Python, a loop is a set of statements that are repeated until a stop condition is met. The while loop syntax includes a stop condition. A while loop works by executing a set of statements until the stop condition is met (Fig. 12.5). A simple example of “while” loop is printing numbers from 0 to 5 in shown in Table 12.11.

In the preceding code (Table 12.14), the variable “a” is initialized at 0. While statements follows with a stop condition of “a” less than 6. While statements comprise two statements, one for printing “a” and the other for incrementing “a” by 1. The value of “a” is printed and incremented from 0 to 1 when the block is first run. The while block’s stop condition is checked before rerunning. The block will rerun if “a” = 1 is less than 6. When “a” equals 6, the loop condition fails and the program will stop. In the absence of a stop condition, or a condition that can never be false, the loop becomes infinite and is executed indefinitely. The Jupyter notebook’s Python kernel has to be restarted in this scenario. If the asterisk on the left side of the cell persists, them it may be in an infinite loop.

The comments accompany each statement in the preceding example (Table 12.14). The stop condition is the last index in the list in this case. The List’s

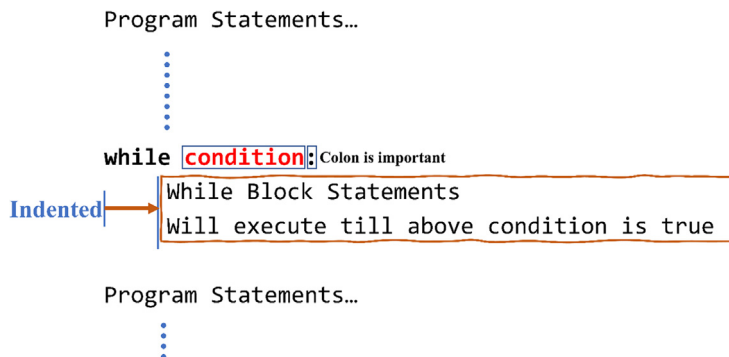


FIG. 12.5

Syntax of a Python while loop.

Table 12.11 Simple while loop.

Code	Output
<pre>a = 0 while a<6: print(a) a = a+1</pre>	0 1 2 3 4 5

last index in the forward direction is 5 and its length is 6, as it contains that many members. Thus, the stop condition is satisfied when the final index is increased by one and equals the length of the ‘total data’ List, i.e. when both are equal to six.

12.8.2 “For” loop

Loops are indefinite loops because they run until a particular condition is false. However, often users need to loop through words in texts or items in lists or through keys and values of a dictionary. In such cases, they require definite loops like “for” loop for doing these tasks of iterating over a sequence datatypes. The “for” loop runs through each item in a set of items. The syntax of a “for” loop is similar as “while” loop: there is a “for” statement and a block of code under that statement (Table 12.12).

The “for” loop iterates over each item in the List and runs the block statements for all items in the List. In simple English, this for loop can be translated as, run the block of code under the “for” loop for every plant in the plants list. In the “for” statement, the “plant” is the variable which changes with every.

12.8.3 Breaking a loop

The “break” command in Python is used to exit a loop early. With a “break” command, the program exits the loop immediately and executes the remaining statements outside the loop block (Table 12.13).

Table 12.12 Simple “for” loop.

Code	Output
<pre>plants = ['Moss', 'Embryophyte', 'Thallophyte','Conifer'] for plant in plants: print(plant)</pre>	Moss Embryophyte Thallophyte Conifer

Table 12.13 Python code snippets showing the breaking a loop before its ending.

Code	Output
<pre>for temp in plants: print(temp) if temp == 'Thallophyte': break</pre>	<pre>Moss Embryophyte Thallophyte</pre>

As the “if” condition is satisfied when “temp” variable equals to “Thallophyte”, the execution of “break” statement blocks the printing of “Conifer”.

12.9 File handling in Python

Until now, our computer’s primary memory has been accessible while constructing and executing statements and programmes. Our computer, as previously stated, has primary and secondary memory, with primary memory being erased when the system is turned off. Secondary memory, such as a hard drive, can save data even after the system has been turned off. Files are often stored on hard drives and have unique paths allocated to them. Biological data is also saved in files with specialized formats such as PDB, networks, and sequence files. As a result, being able to access and manipulate files is crucial for a research project. Files are also used to store and share enormous amounts of data, which is why learning how to read a file in Python is vital.

The first step in file handling is to use Python’s built-in “open ()” function to open a file. Then instruct the operating system to identify and verify the presence of the file by calling the open () function.

Code:

```
f = open('myfile.txt') # If file is in current directory
f = open('C:\Python33\Scripts\myfile.txt') # opening file with Exact Path
```

When only the filename is specified, it is assumed that the file is in the same folder as Python. As illustrated in the second line of code, we may also indicate the file’s location by its full path.

12.9.1 Specify file mode

There are eight types of modes for file operation (Table 12.14). The FASTA file format is used to contain both gene IDs and sequences. After the identifier, the ID line/first line can have further information. It always starts with a “>.” The order of the genes is written after the first line. In the example below (Table 12.15), The targeted file named “myfile.fasta” was opened and read which is located in the current directory or folder. The following is the file’s content:

```
>ID001 (gene abcb1)
GATATGATGCCGTCACTA
GTTTACCGCTGGTAACTG
```

Here “open ()” is a class which deals with operations of files (Table 12.15). The “open” mode is initiated with the name or location of the file. This class has a method called read () which reads all contents of the file. Open handler has a method for returning the List of all the lines known as readlines (). Users can write within a file by opening it in write mode and utilizing the handler class open () and write () function (Table 12.16).

The new file will be saved and can be found in the current folder or directory. The contents of the file will be:

```
>ID002 (gene TLA)
ATGCTTTGGCCAAATTGG
GGTTCATGGTCATGC
TGCTGATC
```

Table 12.14 Modes of file operation in Python.

Character	Mode	Description
“r”	Read (default)	Open a file for read only
“w”	Write	Open a file for write only (overwrite)
“a”	Append	Open a file for write only (append)
“r+”	Read + Write	Open a file for both reading and writing
“x”	Create	Create a new file
“t”	Text (default)	Read and write strings from and to the file.
“b”	Binary	Read and write bytes objects from and to the file. This mode is used for all files that don’t contain text (e.g., images).

Table 12.15 Reading from a file using Python.

Code	Output
<pre>f = open('my_file.fasta') print(f.read())</pre>	<pre>>ID001 (gene abcb1) GATATGATGCCGTCAC TA GTTTACCGCTGGTAACTG</pre>

Table 12.16 Writing inside a file.

Code	Output
<pre>f = open('my_new_file.fasta', 'w') f.write('>ID002 (gene TLA)\n') lines = ['ATGCTTTGGCCAAATTGG\n', 'GGTTCCATGGTCATGC\n', 'TGCTGATC'] f.writelines(lines)</pre>	

When a file is opened, it must be closed with the `close ()` methods of the “f” object. `f.close ()` will close a file.

12.10 Importing functions

A script with the .py extension is a Python module. Python modules can be written relatively quickly. Create a file containing valid Python code, then give it a name and the .py extension. Python’s modules make it easier to modularize programming. When creating Python applications, modules may be accessed by using the “import” keyword. Program will be more dependable and effective by using modules.

For practically every programming task, including web development, database construction, image analysis, data science, statistics, machine learning, and more, Python packages and modules are available. Libraries are a collection of functions, whereas packages are a collection of modules. Over 227,607 Python packages are presently available in the Python Package Index (PyPI), which was created to make life simpler for developers. Using the “pip” installer, we may install any of the PyPI packages that are offered. Both the “conda” and “pip” installers are included in the Python anaconda distribution. Open the terminal or command prompt and run “pip install PackageName” or, for the Anaconda distribution, “conda

install `PackageName`” to install a package. Numerous preloaded packages for data science applications are included in the Python Anaconda distribution. Using the keywords “from” and “import,” we may call the modules once the packages have been installed.

12.10.1 Running a t-test in Python

The relationship between the means of two groups may be deduced using an inferential statistic known as a t-test. T-tests are used when data sets have a normal distribution but unknown variances, such as the a set produced by flipping a coin 100 times.

Assume a pharmaceutical company is testing a brand-new medicine. One group of patients is given the treatment, while another, the “control group,” is given a placebo. A group is given a placebo, which is a substance that doesn’t work as a drug, to see how another group reacts to the real drug. After the drug trial, those in the control group who were given a placebo said their average life expectancy went up by 3 years, while those in the group that got the new therapy said it went up by 4 years. Initial trials indicate that the drug is effective. The observation may, however, have been made by chance. T-test can be used to determine if the findings are consistent across the board.

Assume that we are conducting an experiment in which we expect to accelerate plant development by changing the concentration of a certain nutrient in one plot (the “treatment plot”) while maintaining the same concentration in another and comparing the results between them. To test the difference in plant height between control and treatment plots, we can measure the height randomly from a few samples in each plot. Even if our nutrient has no effect on the plants in the treatment plot, we can still identify random variances in our data (plants never grow at precisely the same rate). If the differences between the plots are too large to be explained by chance, we may use the t-test to find out ([Table 12.17](#)).

This yields two numbers: a T statistic indicating how different our plant sizes are between treatments and a *P*-value indicating the likelihood of obtaining such a large difference by chance if our treatment was ineffective.

Even though the plants in our treatment plot grew larger on average, this t-test result indicates that the difference could easily be due to chance. Because *P* is around 0.03, there is a 3% chance that this result is a coincidence. Differences with a less than 5% chance of occurring by chance are considered statistically significant. So our difference in this example is statistically significant.

12.10.2 Make a simple scatterplot in matplotlib

To attract mates, male frogs frequently make loud calls. Assume we were studying frog calls and wanted to see if larger frogs made longer calls than smaller frogs. Assume we measured the length and size of several frogs’ calls. We’d like to create a

Table 12.17 T-test in Python using scipy library.

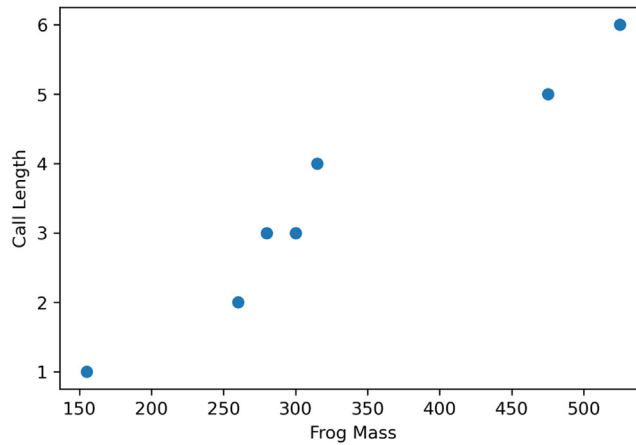
Code	Output
<pre>#Finding the stats module in scipy and importing #its ttest function. from scipy.stats import ttest_ind # control and treatment plot in centimetres. control_plant_sizes = [4,7,8,3,4,2,2] treatment_plant_sizes = [5,4,10,8,5,12,13] #Run a t-test on the results ttest_ind(control_plant_sizes,treatment_plant_s izes)</pre>	<pre>Ttest_indResult(stat istic=- 2.35900095298480, pvalue=0.03611687624 692063)</pre>

graph that depicts the relationship between frog size and call length. A scatterplot, on the other hand, could be a good way to visualize this data.

A scatterplot of frog sizes and frog call durations may be generated by importing the appropriate functions, which include labeling the scatterplot, x and y axes from matplotlib library of python. Matplotlib is a low level and popular graph plotting library in python ([Table 12.18](#) and [Fig. 12.6](#)).

Table 12.18 Plotting scatter plot in Python using matplotlib library.

Code
<pre>from matplotlib.pyplot import scatter, xlabel, ylabel # frog sizes in grams frog_sizes = [155,475,260,525,300,280,315] # call lengths of those same frongs, in seconds frog_call_lengths = [1,5,2,6,3,3,4] scatter(frog_sizes,frog_call_lengths) xlabel("Frog Mass") ylabel("Call Length")</pre>

**FIG. 12.6**

Scatterplot plotted using matplotlib library of Python, where x denotes frog mass and y axis denotes call lengths.

12.10.3 Running a simple linear regression in Python

The t-test function in the previous section came from the `scipy.stats` module, which includes a wide range of statistical analyses. The `pearsonr` function, which performs Pearson regression, is an example of this. This compares the values of two sets of numbers to see if there is any relationship between them. Thus, while t-tests can be used to compare values within a category, Pearson regression can be used to compare two continuous variables.

We can use a Pearson regression to see if larger frogs have longer calls now that we know how long their calls are ([Table 12.19](#)).

12.11 Data handling

Pandas is a high-level data manipulation and analysis package. It is written in Python and is available for free to anyone. Python's pandas library makes it simple to load, format, edit, and inspect data. It is a popular option among developers and is utilized for a variety of data science projects. Pandas are fantastic since they are simple to use and operate quickly. Users can process up to 10 million rows of data without issue. A pandas DataFrame is simple to modify since one can simply add or delete columns, slice it, index it, or deal with missing data. Using pandas makes it easy to clean, modify, and analyze data. One can delve into a dataset stored on their computer as a comma-separated values (CSV) file. Pandas will read the CSV file and convert information to a DataFrame, which is similar to a tables in excel from which the user can perform actions such as:

Table 12.19 Regression analysis using Python.

Code	Output
<pre>#Import the pearsonr function from scipy.stats import pearsonr #Run pearson regression R2, pvalue pearsonr(frog_sizes,frog_call_lengths) #Print the results print("R2:",R2) print("pvalue:",pvalue)</pre>	<pre>R2: = 0.9704929909196073 pvalue: 0.000282726810601368 7</pre>

- Answer queries like “What is the average, median, highest, and lowest value for each column?” via statistical analysis.
- Is there anything that connects rows A and B?
- How can data in a column frequently come apart?
- Remove any incorrect information and sort the data by selecting which rows or columns to examine.
- Use Matplotlib to transform the data into something visual. Anything can be drawn, from lines and bars to bubbles and histograms.
- Return the modified data to a CSV file, a database, or another storage location.

What has been covered in this chapter is only the tip of the iceberg in terms of what Python can do. In any event, the ideas presented here may inspire everyone to experiment with one of their own data sets. It’s simple to go to libraries like Matplotlib, seaborn, or scipy from here, which may be used to visualize data, conduct statistical tests on it, or fit data to mathematical models.

The advancements in the Python language are too numerous to describe in this brief introduction. Users will need to learn more about algorithms, mathematical modeling, data visualization, and other topics. Instead of merely undertaking random programming challenges, if readers want to develop anything important with the language, it may provide them more drive to stay focused and find all of the necessary ingredients. Learning to program computers is an important ability that can help us find better employment and deal with modern time’s issues. Those interested in learning Python will find a wealth of high-quality resources. Some of these have already been mentioned, and [Table 12.20](#) shows a list of available resources.

Table 12.20 Resources for learning Python.

Source type	Address	Description
Python	https://www.python.org/	Official python website
Codes of published research articles	http://hplgit.github.io/bioinf-py/doc/pub/html/index.html	Illustrating python via examples from bioinformatics
	https://paperswithcode.com/	Open resource with machine learning papers, code, datasets, methods and evaluation tables.
Biopython	https://biopython.org/ ; http://biopython.org/DIST/docs/tutorial/Tutorial.html	The Biopython project is an international collaboration of scientists working to create open-source python programmes for use in computational molecular biology.
Visualization of Genomic data	https://gangcaolab.github.io/CoolBox/index.html	Users can use CoolBox to create high-quality visualization plots and analyze their data in a flexible, customizable, and user-friendly manner. Molecular computational biology

References

Ekmekci, B., McAnany, C.E., Mura, C., 2016. An introduction to programming for bioscientists: A python-based primer. PLoS Comput. Biol. 12, e1004867. <https://doi.org/10.1371/JOURNAL.PCBI.1004867>.

Hasija, Y., Chakraborty, R., 2021. Hands on Data Science for Biologists Using Python. CRC Press.

Marx, V., 2013. The big challenges of big data. Nature 7453 (498), 255–260. <https://doi.org/10.1038/498255a>.

Python 3.0 Release | Python.org [WWW Document], (n.d.). URL <https://www.python.org/download/releases/3.0/> (accessed 8.21.2022).

van Rossum, G., Drake, F.L., 2009. Python 3 Reference Manual. CreateSpace, Scotts Valley, CA.